SET 25

PART A — Short questions and answers

Q1. Summarize how an ER diagram supports the conceptual design of a vehicle service database.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Interpret the statement: UPDATE SERVICE_JOB SET Cost = Cost + 500 WHERE ServiceType = 'Engine';

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Summarize the insertion, update and deletion anomalies caused by redundancy.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Explain how the ALL operator works in a subquery comparison.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Compare the COUNT and SUM functions using service job records.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q6. Describe a row-level trigger.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q7. Summarize the situations in which a document database is a suitable choice.

Answer: Document databases allow documents to have different fields, making them useful when records vary.

Q8. Explain the purpose of the sort() method.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Interpret the MongoDB query: db.vehicles.find({Brand: "Honda"}).sort({PurchaseYear: -1}).limit(2)

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Explain how the pattern (m:Mechanic)-[:SERVICED]->(v:Vehicle) is read from left to right.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: An automobile service center maintains a Vehicle Service Center Management System to store customer

1(a) ER design:

• **CUSTOMER**(CustomerID, CustomerName, Phone, City); primary key: **CustomerID**.

• **MECHANIC**(MechanicID, MechanicName, Specialization); primary key: **MechanicID**.

• **SERVICE_JOB**(JobID, CustomerID, MechanicID, VehicleNo, ServiceDate, ServiceType, Cost); primary key: **JobID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE CUSTOMER (
  CustomerID NUMBER PRIMARY KEY,
  CustomerName VARCHAR2(100),
  Phone VARCHAR2(100),
  City VARCHAR2(100)
);
```

```
CREATE TABLE MECHANIC (
  MechanicID NUMBER PRIMARY KEY,
  MechanicName VARCHAR2(100),
  Specialization VARCHAR2(100)
);
CREATE TABLE SERVICE_JOB (
  JobID NUMBER PRIMARY KEY,
  CustomerID NUMBER,
  MechanicID NUMBER,
  VehicleNo VARCHAR2(100),
  ServiceDate DATE,
  ServiceType VARCHAR2(100),
  Cost NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM SERVICE_JOB x
JOIN CUSTOMER a ON x.CustomerID = a.CustomerID
JOIN MECHANIC b ON x.MechanicID = b.MechanicID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```